Natural Computing SoSe25

Exercise Sheet 2 — May 22, 2025

Thomas Gabor, Maximilian Zorn

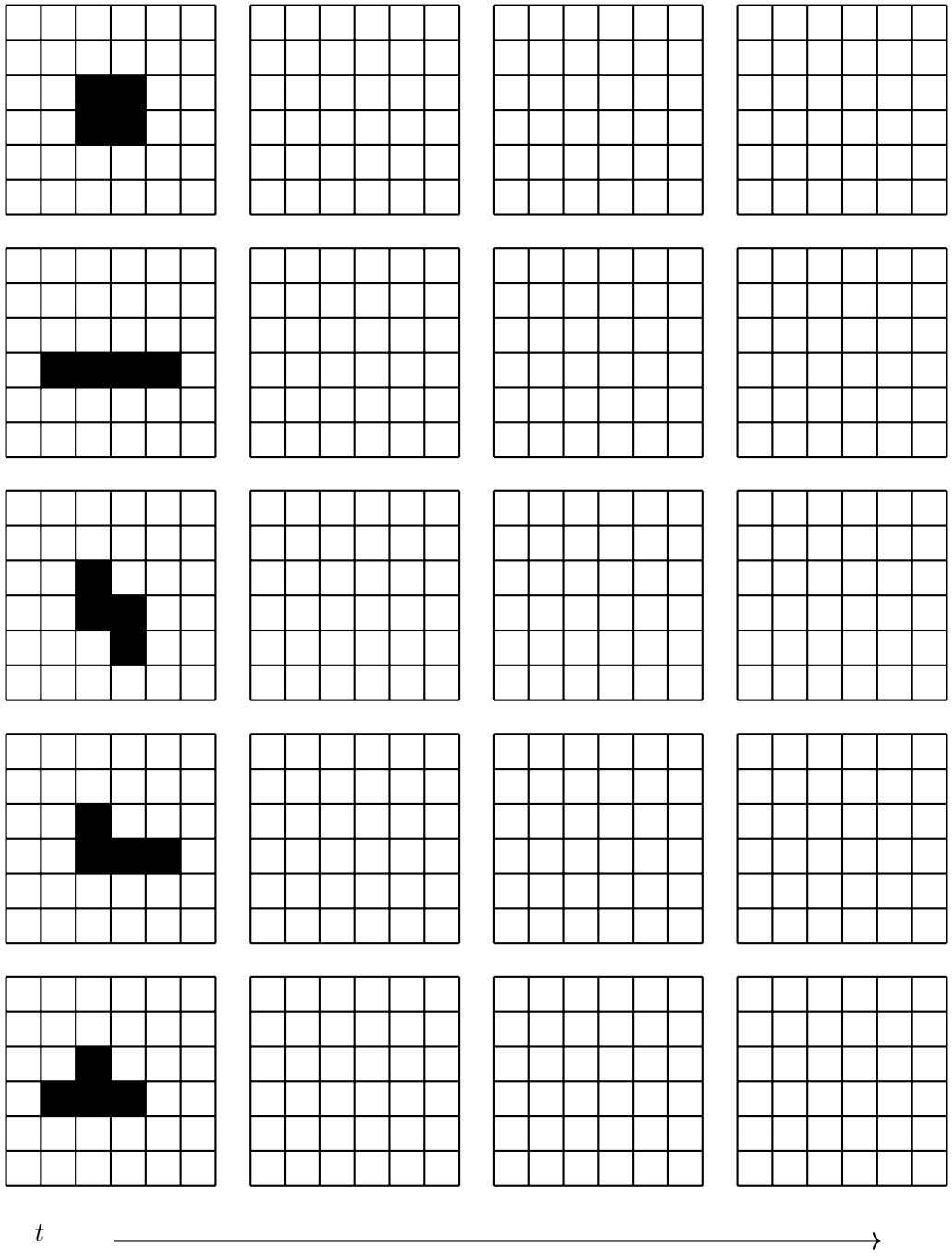
1 Life in a Cellular Automaton

(i) In the lecture we have discussed that the Game of Life is a special instance of a cellular automaton. As discussed in Definition 1 on the definition sheet, a cellular automaton is specified by providing a graph G including a neighborhood function *neighborhood*, a step function f, and an initial state x_0. Specify a cellular automaton that implements the Game of Life on an infinite playing field with the Game of Life starting state S given via

$$S(v) = \begin{cases} \textit{alive} & \text{if } v \in \{(0,0),(0,1),(1,0),(1,1)\}, \\ \textit{dead} & \text{otherwise.} \end{cases}$$

What pattern does the starting state S implement?

(ii) With the standard rules of *Life* (B3/S23) and the standard Moore neighborhood around every cell (cf. Definition 3 in the definition sheet), evolve the following patterns by hand for at most three steps, until they cannot be evaluated any further or their state does not change anymore. What are the final states called?


t

(iii) Drawing and evolving every pattern by hand on a piece of paper is very tedious. In the very first unveiling of Life in the *Scientific American in 1970*[1], columnist Martin Gardner recommends:

To play life you must have a fairly large checkerboard and a plentiful supply of flat counters of two colors. (Small checkers or poker chips do nicely.) An Oriental "Go" board can be used if you can find flat counters that are small enough to fit within its cells. (Go stones are unusable because they are not flat.) It is possible to work with pencil and graph paper but it is much easier, particularly for beginners, to use counters and a board.

Since then simulation of Life has come a long way (e.g., `https://conwaylife.com/`). As inputting larger initial pattern quickly becomes impractical, one could record Life patterns in Run Length Encoding (RLE). (See the wiki at conwaylife.com[2] for the definition.) On the conwaylife.com simulator under the *'Show Advanced Options'* button you can input and run such encodings. Encode the initial patterns of exercise above with RLE and run the simulation.

2 The 'Game of Life' Game

In your exercise you will find attached another `pygame` template in the `gol_game.py` file. The structure is similar to the 1d CA game from Exercise 1 but with a few tweaks to bring the game into the second dimension. If you run the initial file you should see a blank board with a generation counter on the top left of the screen.

(i) In the `*** marked sections ***`, code in the process for finding the *surrounding* cells (in the `moore_surroundings()` function) and for setting a *glider* pattern on the screen (in the `glider_pattern()` function).

(ii) (Bonus) In the `*** marked sections ***` in the `rle_pattern()` function, code in the process for decoding and drawing a GoL run-length encoded string to the board.
Hint: Read the comment carefully, it alreay hints at one specific solution...

[1] `https://www.ibiblio.org/lifepatterns/october1970.html`
[2] `https://conwaylife.com/wiki/Run_Length_Encoded`

3 Beyond the Game of Life

You will also find attached the `beyond_gol_game.py` file, which is a generalized version of a 2d CA. Apart from two imports, the code will already run as it is.

(i) Consider the `beyond_gol_game.py` code implementation and compare it to the (your) `gol_game.py` version. What extensions compared to the standard *Game of Life* are implemented? Briefly reason about their usefulness.

(ii) The represented system you see implemented here is often referred to as ***Primordia***. Adjust our definition from the lecture to match this generalized, normalized version of the game, i.e., fill in the definition below.
You may use the functions `clip:` $\mathbb{R} \to [0;1]$ and `growth:` $\mathbb{R} \to \mathbb{R}$ as you see fit.

Exercise Definition (Primordia) Let $G = (V, E)$ be a graph with vertices V and (undirected) edges $E \subseteq V \times V$. We define *neighborhood* $: V \to \wp(V)$ via *neighborhood*$(v) = \{w \mid (v, w) \in E\}$ as a topology for G. A state $x \in \mathcal{X}$ is a mapping of vertices

to ,

i.e., the state space $\mathcal{X}$ is given

via .

Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$|v|_{x_t} =$.

In Primordia, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically

via ,

for all $v \in V$. A tuple $(G, x_S,)$ is called an instance of the *Primordia* for initial state $x_S \in \mathcal{X}$.

(iii) What parameters would you have to change in the `beyond_gol_game.py` file to return back to our canonical definition of *Game of Life*?

(iv) (Bonus) Beyond the world of *Primordia* we can extend the concept of continuous states to work with continuous time and even space. In fact, quite recently the concept of ***Lenia***[3] has done exactly that, renewing the scientific interest in 2d CA games. You can explore the (implemented) concept online[4]. What differences can you spot to the *Game of Life* and *Primordia*?

[3] `https://chakazul.github.io/lenia.html`

[4] `https://colab.research.google.com/github/OpenLenia/Lenia-Tutorial/blob/main/Tutorial_From_Conway_to_Lenia.ipynb`